

Chap.5 합성곱 신경망 I (Part 2 - 5.3~)

5.3 전이 학습

5.3.1 특성 추출 기법

특성 추출 예제 코드

5.3.2 미세 조정 기법

5.4 설명 가능한 CNN

5.4.1 특성 맵 시각화

특성 맵 시각화 예제 코드

5.5 그래프 합성곱 네트워크

5.5.1 그래프란

5.5.2 그래프 신경망

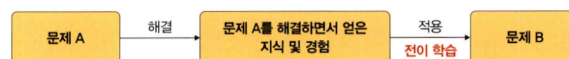
5.5.3 그래프 합성곱 네트워크

5.3 전이 학습

일반적으로 합성곱 신경망 기반의 딥러닝 모델을 제대로 훈련시키려면 많은 양의 데이터가 필요함. 하지만 충분히 큰 데이터셋을 확보하기 위해서는 많은 돈과 시간이 필요하기 때문에, 얻는 것이 쉽지 않음.

이러한 현실적 어려움을 해결한 것이 전이 학습(transfer learning).

- 전이학습이란?



ImageNet처럼 아주 큰 데이터셋을 써서 훈련된 모델의 가중치를 가져와 우리가 해결하려는 과제에 맞게 보정해서 사용하는 것을 의미.

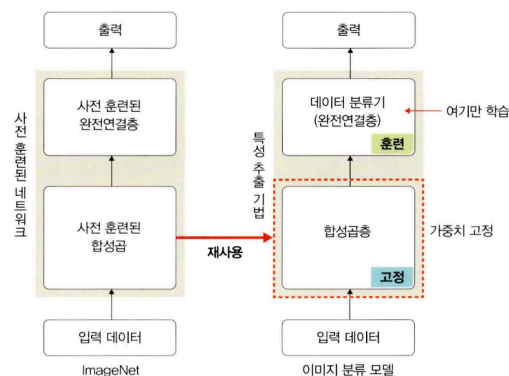
이때 아주 큰 데이터셋을 사용하여 훈련된 모델을 사전 훈련된 모델(네트워크)이라고 함.

→ 비교적 적은 수의 데이터를 가지고도 우리가 원하는 과제 해결 가능!

- 전이 학습을 위한 방법으로는 1) 특성 추출과 2) 미세 조정 기법이 있음.

5.3.1 특성 추출 기법

특성 추출(feature extractor)은 ImageNet 데이터셋으로 사전 훈련된 모델을 가져온 후 마지막에 fully connected layer 부분만 새로 만들. 즉, 학습할 때는 마지막 fully connected layer(이미지의 카테고리를 결정하는 부분)만 학습하고 나머지 계층들은 학습되지 않도록 함.



특성 추출은 이미지 분류를 위해 두 부분으로 구성됨.

- 합성곱층 : 합성곱층과 풀링층으로 구성
- 데이터 분류기(완전연결층) : 추출된 특성을 입력받아 최종적으로 이미지에 대한 클래스를 분류하는 부분

사전 훈련된 네트워크의 합성곱층(가중치 고정)에 새로운 데이터를 통과시키고, 그 출력을 데이터 분류기에서 훈련시킴.

여기에서 사용 가능한 이미지 분류 모델 : Xception, Inception V3, ResNet50, VGG16, VGG19, MobileNet

특성 추출 예제 코드

▼ c.f. 특성 추출을 위한 라이브러리

```
pip install opencv-python
```

OpenCV는 Open Source Computer Vision Library의 약어로 오픈 소스 컴퓨터 비전 라이브러리.

```
# 라이브러리 호출
import os
import time
import copy
import glob
import cv2 # OpenCV 라이브러리
import shutil

import torch
import torchvision # CV 용도의 패키지
import torchvision.transforms as transforms # 데이터 전처리를 위해 사용되는 패키지
import torchvision.models as models # 다양한 파이토치 네트워크를 사용할 수 있도록 도와주는 패키지
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

import numpy as np
import matplotlib.pyplot as plt
```

```
# 이미지 데이터 전처리 방법 정의
data_path = '/Users/eunseoshin/DL-pytorch/chap05/data/catanddog/train' # 이미지 데이터가 위치한 경로 지정

transform = transforms.Compose(
    [
        transforms.Resize([256, 256]),
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor()
    ]
) # 설명(1)

train_dataset = torchvision.datasets.ImageFolder(
    data_path,
    transform=transform
) # 설명(2)

train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=32,
    num_workers=8,
    shuffle=True
) # 설명(3)
```

```
print(len(train_dataset))
```

▼ 코드 실행 결과

385

• 설명 (1)

`torchvision.transform` 은 이미지 데이터를 변환해 모델(네트워크)의 입력으로 사용할 수 있게 변환해줌. 이때 사용하는 파라미터는 다음과 같다.

```
transform = transforms.Compose([
    transforms.Resize([256, 256]),
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor()])
```

- `Resize` : 이미지의 크기를 조정. (256 x 256 크기로 이미지 데이터 조정)

- `RandomResizedCrop` : 이미지를 랜덤한 크기 및 비율로 자름

▼ c.f. `Resize` 와 `RandomResizedCrop` 용도의 차이

둘 다 이미지를 자르는데 사용하지만 그 용도는 다름.

`Resize` 는 합성곱층을 통과하기 위해 이미지 크기를 조정하는 전처리 과정.

`RandomResizedCrop` 은 데이터 확장 용도로 사용됨. 이미지를 랜덤한 비율로 자른 후 데이터 크기를 조정함.

- `RandomHorizontalFlip` : 이미지를 랜덤하게 수평으로 뒤집음
- `ToTensor` : 이미지 데이터를 텐서로 변환

• 설명 (2)

`dataset.ImageFolder` 는 `DataLoader`가 데이터를 불러올 대상(혹은 경로)과 방법(transform)(혹은 전처리)을 정의하며, 사용하는 파라미터는 다음과 같다.

```
train_dataset = torchvision.datasets.ImageFolder(
    data_path, transform=transform)
```

- 첫 번째 파라미터 : 불러올 데이터가 위치한 경로
- `transform` : 이미지 데이터에 대한 전처리

• 설명 (3)

데이터로더는 데이터를 불러오는 부분으로 앞에서 정의한 `ImageFolder(train_dataset)` 을 데이터로더에 할당함.

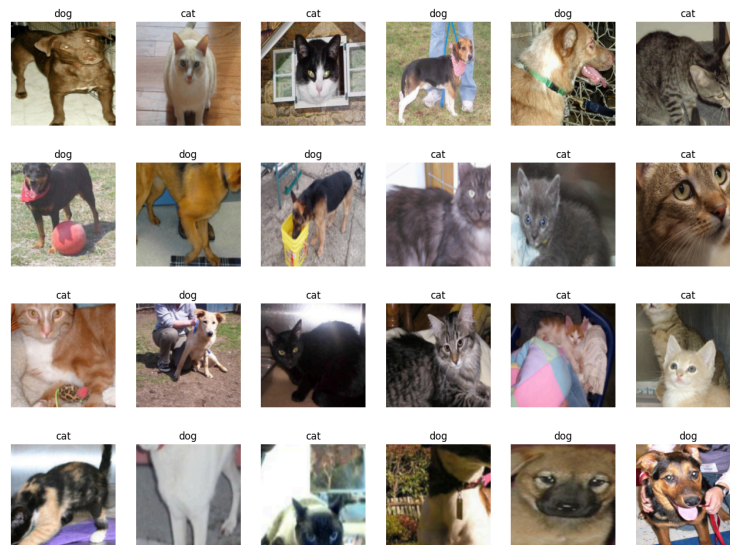
데이터로더에서 사용하는 파라미터는 다음과 같다.

```
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=32,
    num_workers=8,
    shuffle=True
)
```

- 첫 번째 파라미터 : 데이터셋 지정
- `batch_size` : 한 번에 불러올 데이터양을 결정하는 배치 크기 설정
- `num_workers` : 데이터를 불러올 때 하위 프로세스를 몇 개 사용할지 결정하는데, 이때 너무 많은 하위 프로세스를 설정하게 되면 오류가 발생하거나 메모리 부족 현상이 발생할 수 있음. (보통 CPU 코어수 이하로 설정하는 것 권장)
- `shuffle` : 데이터를 무작위로 섞을지 지정. `shuffle=True` 로 설정하면 데이터를 무작위로 섞어서 랜덤으로 불러옴.

```
# 학습에 사용될 이미지 출력
samples, labels = next(iter(train_loader)) # 설명(1)
classes = {0:'cat', 1:'dog'} # 개와 고양이에 대한 클래스로 구성
fig = plt.figure(figsize=(16,24))
for i in range(24): # 24개의 이미지 데이터 출력
    a = fig.add_subplot(4,6,i+1)
    a.set_title(classes[labels[i].item()]) # 레이블 정보(클래스)를 함께 출력
    a.axis('off')
    a.imshow(np.transpose(samples[i].numpy(), (1,2,0))) # 설명(2)
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0)
```

▼ 코드 실행 결과



• 설명 (1)

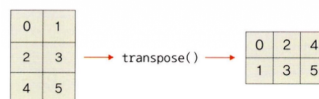
반복자(iterator, for 구문과 같은 효과)를 사용하려면 `iter()` 와 `next()` 가 필요함.

`iter()` 는 전달된 데이터의 반복자를 꺼내 반환하며, `next()` 는 그 반복자가 다음에 출력해야 할 요소를 반환. 즉, `iter()` 로 반복자를 구하고, 그 반복자를 `next()` 에 전달해 차례대로 꺼낼 수 있음.

코드에서 반복자는 `train_loader`가 되기 때문에 `train_loader`에서 `samples`와 `labels`의 값을 순차적으로 꺼내서 저장함. (`train_loader`에서 데이터를 하나씩 꺼내 오겠다는 의미)

• 설명 (2)

`np.transpose` 는 그림과 같이 행과 열을 바꿈으로써 행렬의 차원을 바꿔줌.



일반적으로 행렬의 차원을 변경하는 이유는 행렬의 내적 연산 때문.

아래 그림과 같이 A라는 행렬과 B라는 행렬의 내적 결과는 C가 됨. 이때 A 행렬의 행과 B 행렬의 열의 수가 같아야 내적 가능. 둘의 행과 열의 수가 달라 내적이 불가능할 때는 `np.transpose()` 나 `np.reshape()` 등으로 차원을 조정해야 함.

곱셈, 덧셈

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

A B C

$$\begin{bmatrix} c_{11} \\ c_{21} \\ c_{31} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} \\ a_{21}b_{11} + a_{22}b_{21} + a_{23}b_{31} \\ a_{31}b_{11} + a_{32}b_{21} + a_{33}b_{31} \end{bmatrix}$$

```
# 사전 훈련된 모델 내려받기
resnet18 = models.resnet18(pretrained=True) # pretrained=True는 사전 학습된 가중치를 사용
하겠다는 의미
```

- ResNet18

ResNet18은 50개의 계층으로 구성된 합성곱 신경망.

ImageNet 데이터베이스의 100만 개가 넘는 영상을 이용해 훈련된 신경망으로 전이학습에 사용되도록 사전 훈련된 모델을 제공함. 하지만 ResNet18은 입력 제약이 매우 크고, 충분한 메모리(RAM)가 없으면 학습 속도가 느릴 수 있는 단점 존재. (자세한 내용은 6장에서..)

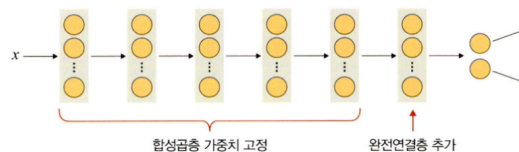
```
# 사전 훈련된 모델의 파라미터 학습 유무 지정
def set_parameter_requires_grad(model, feature_extracting=True):
    if feature_extracting:
        for param in model.parameters():
            param.requires_grad = False # 설명

set_parameter_requires_grad(resnet18)
```

- `param.requires_grad = False`

역전파 중 파라미터들에 대한 변화를 계산할 필요가 없음을 나타냄. 즉, 모델의 일부를 고정하고 나머지를 학습하고자 할 때 `requires_grad = False`로 설정함. 이때 모델의 일부는 합성곱층(convolutional layer)과 풀링(pooling)층을 의미함.

내려받은 ResNet18의 마지막 부분에 완전연결층 추가. 추가된 완전연결층은 개와 고양이 클래스를 분류하는 용도로 사용됨.



```
# ResNet18에 완전연결층 추가
resnet18.fc = nn.Linear(512, 2) # 2는 클래스가 두 개라는 의미
```

```
# 모델의 파라미터 값 확인
for name, param in resnet18.named_parameters(): # model.named_parameters()는 모델에 접근
하여 파라미터 값들을 가져올 때 사용
    if param.requires_grad:
        print(name, param.data)
```

▼ 코드 실행 결과

```
fc.weight tensor([[-0.0114,  0.0340,  0.0230, ..., -0.0273,  0.0412,  0.0387],
                  [-0.0201, -0.0057,  0.0157, ..., -0.0253,  0.0141,  0.0060]])
fc.bias tensor([-0.0088,  0.0087])
```

파라미터는 weight와 bias가 사용되고 있음.

```
# 모델 객체 생성 및 손실 함수 정의
model = models.resnet18(pretrained=True) # 모델의 객체 생성

for param in model.parameters(): # 모델의 합성곱층 가중치 고정
    param.requires_grad = False

model.fc = torch.nn.Linear(512, 2)
for param in model.fc.parameters(): # 완전연결층은 학습
    param.requires_grad = True

optimizer = torch.optim.Adam(model.fc.parameters())
cost = torch.nn.CrossEntropyLoss() # 손실 함수 정의
print(model)
```

▼ 코드 실행 결과

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    )
    ...
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=512, out_features=2, bias=True)
)
```

```
# 모델 학습을 위한 함수 생성
def train_model(model, dataloaders, criterion, optimizer, device, num_epochs=13,
                is_train=True):
    since = time() # 컴퓨터의 현재 시각을 구하는 함수
    acc_history = []
    loss_history = []
    best_acc = 0.0

    for epoch in range(num_epochs): # 에포크(13)만큼 반복
        print('Epoch {}/{}'.format(epoch, num_epochs-1))
        print('-' * 10)

        running_loss = 0.0
        running_corrects = 0

        for inputs, labels in dataloaders: # 데이터로더에 전달된 데이터만큼 반복
            inputs = inputs.to(device)
```

```

        labels = labels.to(device)

        model.to(device)
        optimizer.zero_grad() # 기울기 0으로 설정
        outputs = model(inputs) # 순전파 학습
        loss = criterion(outputs, labels)
        _, preds = torch.max(outputs, 1)
        loss.backward() # 역전파 학습
        optimizer.step()

        running_loss += loss.item() * inputs.size(0) # 출력 결과와 레이블의 오차를 계
산한 결과를 누적하여 저장
        running_corrects += torch.sum(preds == labels.data) # 출력 결과와 레이블이 동
일한지 확인한 결과를 누적하여 저장

        epoch_loss = running_loss / len(dataloaders.dataset) # 평균 오차 계산
        epoch_acc = running_corrects.float() / len(dataloaders.dataset) # 평균 정확도 계
산

        print('Loss: {:.4f} Acc: {:.4f}'.format(epoch_loss, epoch_acc))

        if epoch_acc > best_acc:
            best_acc = epoch_acc

        acc_history.append(epoch_acc.item())
        loss_history.append(epoch_loss)
        torch.save(model.state_dict(), os.path.join('/Users/eunseoshin/DL-pytorch/chap
05/data/catanddog/', '{0:0=2d}.pth'.format(epoch))) # 모델 재사용을 위해 저장해두기
        print()

        time_elapsed = time.time() - since # 실행 시간(학습 시간)을 계산
        print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elaps
ed % 60))
        print('Best Acc: {:.4f}'.format(best_acc))
        return acc_history, loss_history # 모델의 정확도와 오차를 반환

```

ResNet18에 추가된 완전연결층은 학습을 하도록 설정. 학습을 통해 얻어지는 파라미터를 옵티마이저에 전달해 최종적으로 모델학
습에 사용.

```

# 파라미터 학습 결과를 옵티마이저에 전달
params_to_update = []
for name, param in resnet18.named_parameters():
    if param.requires_grad == True:
        params_to_update.append(param) # 파라미터 학습 결과를 저장
        print("\t", name)

optimizer = optim.Adam(params_to_update) # 학습 결과를 옵티마이저에 저장

```

▼ 코드 실행 결과

```

fc.weight
fc.bias

```

완전연결층의 어떤 파라미터들이 옵티마이저로 전달되는지를 보여줌.
결과) weight와 bias 값들이 업데이트되고 옵티마이저에 전달될 것이다.

```
# 모델 학습
device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
criterion = nn.CrossEntropyLoss() # 손실함수 지정
train_acc_hist, train_loss_hist = train_model(resnet18, train_loader, criterion, optimizer, device)
```

▼ 코드 실행 결과

Epoch 0/12 ----- Loss: 0.5398 Acc: 0.7299	Epoch 7/12 ----- Loss: 0.2350 Acc: 0.9117
Epoch 1/12 ----- Loss: 0.4292 Acc: 0.7948	Epoch 8/12 ----- Loss: 0.2445 Acc: 0.8961
Epoch 2/12 ----- Loss: 0.3820 Acc: 0.8234	Epoch 9/12 ----- Loss: 0.2489 Acc: 0.8987
Epoch 3/12 ----- Loss: 0.3382 Acc: 0.8312	Epoch 10/12 ----- Loss: 0.2032 Acc: 0.9143
Epoch 4/12 ----- Loss: 0.3307 Acc: 0.8519	Epoch 11/12 ----- Loss: 0.2241 Acc: 0.8883
Epoch 5/12 ----- Loss: 0.2327 Acc: 0.8961	Epoch 12/12 ----- Loss: 0.1733 Acc: 0.9403
Epoch 6/12 ----- Loss: 0.2328 Acc: 0.9221	Training complete in 9m 33s Best Acc: 0.940260

약 94%로 상당히 높은 정확도를 보임. → 훈련 데이터로는 학습 good!

- 모델 훈련을 위해 전달되는 파라미터 : (모델, 학습 데이터, 손실함수, 옵티마이저, 장치(CPU or GPU))

```
# 테스트 데이터 호출 및 전처리
test_path = '/Users/eunseo shin/DL-pytorch/chap05/data/catanddog/test'

transform = transforms.Compose(
    [
        transforms.Resize(224),
        transforms.CenterCrop(224),
        transforms.ToTensor()
    ]
)

test_dataset = torchvision.datasets.ImageFolder(
    root=test_path,
    transform=transform
)

test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=32,
    num_workers=1,
    shuffle=True
)

print(len(test_dataset))
```

▼ 코드 실행 결과

98

```
# 테스트 데이터 평가 함수 생성
def eval_model(model, dataloaders, device):
    since = time.time()
    acc_history = []
    best_acc = 0.0

    saved_models = glob.glob('/Users/eunseoshin/DL-pytorch/chap05/data/catanddog/' +
                              '*.pth') # 설명(1)
    saved_models.sort() # 불러온 .pth 파일들을 정렬
    print('saved_model', saved_models)

    for model_path in saved_models:
        print('Loading model', model_path)

        model.load_state_dict(torch.load(model_path))
        model.eval()
        model.to(device)
        running_corrects = 0

        for inputs, labels in dataloaders: # 테스트 반복
            inputs = inputs.to(device)
            labels = labels.to(device)

            with torch.no_grad(): # autograd를 사용하지 않겠다는 의미
                outputs = model(inputs) # 데이터를 모델에 적용한 결과를 outputs에 저장

            _, preds = torch.max(outputs.data, 1) # 설명(2)
            preds[preds >= 0.5] = 1 # torch.max로 출력된 값이 0.5보다 크면 올바르게 예측
            preds[preds < 0.5] = 0 # torch.max로 출력된 값이 0.5보다 작으면 틀리게 예측
            running_corrects += preds.eq(labels).int().sum() # 설명(3)

    epoch_acc = running_corrects.float() / len(dataloaders.dataset) # 테스트 데이터
    의 정확도 계산
    print('Acc: {:.4f}'.format(epoch_acc))

    if epoch_acc > best_acc:
        best_acc = epoch_acc
        acc_history.append(epoch_acc.item())
        print()

    time_elapsed = time.time() - since
    print('Validation complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print('Best Acc: {:.4f}'.format(best_acc))

    return acc_history # 계산된 정확도 반환
```

• 설명 (1)

`glob` 은 현재 디렉터리에서 원하는 파일들만 추출하여 가져올 때 사용. 지정 경로에서 pth 확장자를 갖는 파일을 가져오라는 의미. (pth 확장자 파일은 훈련 데이터로, 모델을 훈련시킬 때 생성된 파일임)

• 설명 (2)

`torch.max` 는 주어진 텐서 배열의 최댓값이 들어 있는 index를 반환하는 함수.

- 설명 (3)

`preds.eq(labels)` 는 preds 배열과 labels가 일치하는지 검사하는 용도로 사용. 또한 뒤에 사용된 `.sum()` 은 모델의 예측 결과와 정답(레이블)이 일치하는 것들의 개수 합을 숫자로 출력함.

```
# 테스트 데이터를 평가 함수에 적용
val_acc_hist = eval_model(resnet18, test_loader, device)
```

▼ 코드 실행 결과

```
saved_model ['Users/eunseoshin/DL-pytorch/chap05/data/catanddog/00.pth', 'Users
Loading model /Users/eunseoshin/DL-pytorch/chap05/data/catanddog/00.pth
/var/folders/4q/3301trc58fz6_vl0nx1b71r00000gn/T/ipykernel_68941/4270964118.py:14
model.load_state_dict(torch.load(model_path))
Acc: 0.9388

Loading model /Users/eunseoshin/DL-pytorch/chap05/data/catanddog/01.pth
Acc: 0.8367

Loading model /Users/eunseoshin/DL-pytorch/chap05/data/catanddog/02.pth
Acc: 0.9286

Loading model /Users/eunseoshin/DL-pytorch/chap05/data/catanddog/03.pth
Acc: 0.9286

Loading model /Users/eunseoshin/DL-pytorch/chap05/data/catanddog/04.pth
Acc: 0.9388

Loading model /Users/eunseoshin/DL-pytorch/chap05/data/catanddog/05.pth
Acc: 0.9286

Loading model /Users/eunseoshin/DL-pytorch/chap05/data/catanddog/06.pth
Acc: 0.9286

Loading model /Users/eunseoshin/DL-pytorch/chap05/data/catanddog/07.pth
Acc: 0.9286

Loading model /Users/eunseoshin/DL-pytorch/chap05/data/catanddog/08.pth
Acc: 0.9388

Loading model /Users/eunseoshin/DL-pytorch/chap05/data/catanddog/09.pth
Acc: 0.9592

Loading model /Users/eunseoshin/DL-pytorch/chap05/data/catanddog/10.pth
Acc: 0.9388

Loading model /Users/eunseoshin/DL-pytorch/chap05/data/catanddog/11.pth
Acc: 0.9082

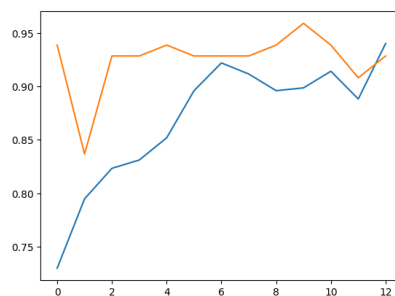
Loading model /Users/eunseoshin/DL-pytorch/chap05/data/catanddog/12.pth
Acc: 0.9286

Validation complete in 1m 26s
Best Acc: 0.9592
```

테스트 데이터 역시 95% 정도의 높은 정확도를 보임.

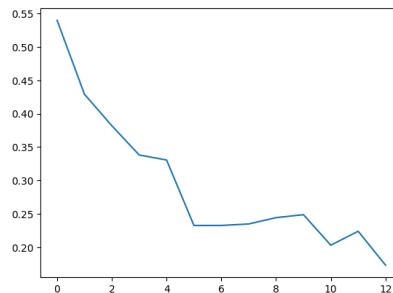
```
# 훈련과 테스트 데이터의 정확도를 그래프로 확인
plt.plot(train_acc_hist)
plt.plot(val_acc_hist)
plt.show()
```

▼ 코드 실행 결과



```
# 훈련 데이터의 오차에 대한 그래프 확인
plt.plot(train_loss_hist)
plt.show()
```

▼ 코드 실행 결과



```
# 예측 이미지 출력을 위한 전처리 함수
def im_convert(tensor):
    image = tensor.clone().detach().numpy() # 설명(1)
    image = image.transpose(1, 2, 0)
    image = image * (np.array((0.5, 0.5, 0.5)) + np.array((0.5, 0.5, 0.5)))
    image = image.clip(0, 1) # 설명(2)
    return image
```

• 설명 (1)

`tensor.clone()` 은 기존 텐서의 내용을 복사한 텐서를 생성하겠다는 의미.

`detach()` 는 기존 텐서에서 기울기가 전파되지 않는 텐서.

→ `tensor.clone().detach()` 는 기존 텐서를 복사한 새로운 텐서를 생성하지만 기울기에 영향을 주지는 않겠다는 의미.

▼ `tensor.clone()`, `tensor.detach()`, `tensor.clone().detach()` 비교

구분	메모리	계산 그래프 상주 유무
<code>tensor.clone()</code>	새롭게 할당	계산 그래프에 계속 상주
<code>tensor.detach()</code>	공유해서 사용	계산 그래프에 상주하지 않음
<code>tensor.clone().detach()</code>	새롭게 할당	계산 그래프에 상주하지 않음

• 설명 (2)

`clip()` 은 입력 값이 주어진 범위를 벗어날 때 입력 값을 특정 범위로 제한시키기 위해 사용.

`image.clip(0, 1)` 은 image 데이터를 0과 1 사이의 값으로 제한하겠다는 의미

▼ e.g.

```
> import numpy as np
> exam = np.array([-1.8, -1.2, -0.7, 0.0, 0.8, 1.4, 1.9])
> print(exam)
[-1.8 -1.2 -0.7  0.  0.8  1.4  1.9]
> print(np.clip(exam, -0.5, 0.5))
[-0.5 -0.5 -0.5  0.  0.5  0.5  0.5]
```

```
# 개와 고양이 예측 결과 출력
classes = {0:'cat', 1:'dog'} # 개와 고양이 두 개에 대한 레이블

dataiter = iter(test_loader) # 테스트 데이터셋을 가져옴
images, labels = next(dataiter) # 테스트 데이터셋에서 이미지와 레이블을 분리하여 가져옴
model.to(device)
output = model(images.to(device))
_, pred = torch.max(output, 1)

fig = plt.figure(figsize=(25,4))
for idx in np.arange(20):
    ax = fig.add_subplot(2, 10, idx+1, xticks=[], yticks=[]) # 설명(1)
    plt.imshow(im_convert(images[idx])) # 이미지 출력을 위해 앞 셀에서 정의한 im_convert 함수 적용
    ax.set_title("{} ({}).format(str(classes[pred[idx].item()]), str(classes[labels[idx].item()])), color=("green" if pred[idx]==labels[idx] else "red")) # 설명(2)
plt.show()
plt.subplots_adjust(bottom=0.2, top=0.6, hspace=0) # 설명(3)
```

▼ 코드 실행 결과



예측이 정확하지 않음. → 훈련 데이터를 더 늘리고, 에포크 횟수도 늘리면 더 좋은 결과를 얻을 수 있을 것임.

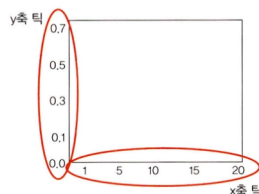
• 설명 (1)

add_subplot은 한 화면에 여러 개의 이미지를 담기 위해 사용. 이때 사용되는 파라미터는 다음과 같다.

```
add_subplot(2, 10, idx+1, xticks=[], yticks=[])
```

- 첫 번째 파라미터 : 행의 수를 의미. 이미지를 두 줄로 출력
- 두 번째 파라미터 : 열의 수를 의미. 한 줄에 열 개의 이미지 출력
- 세 번째 파라미터 : 인덱스를 의미. 행과 열을 기준으로 순차적으로 이미지를 출력
- `xticks=[], yticks=[]` : 틱(tick)을 삭제하겠다는 의미.

▼ c.f. 틱(tick)이란?

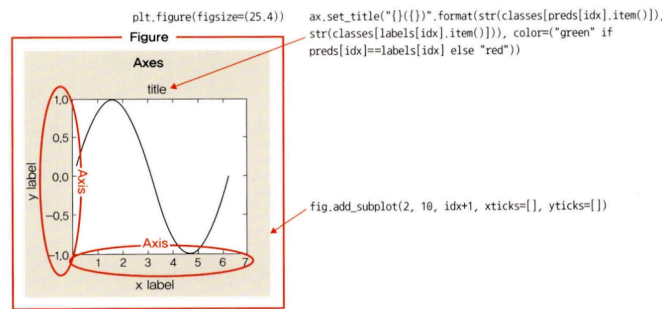


• 설명 (2)

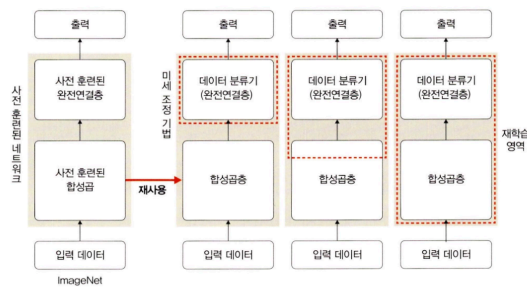
`classes[pred[idx].item()]` 은 `preds[idx].item()` 값이 `classes`로 정의된 '0'과 '1' 중 어떤 값을 갖는지 판별하겠다는 의미. 즉, `classes[pred[idx].item()]` 값이 0이면 고양이, 1이면 개로 출력됨.

• 설명 (3)

아래 그림과 같이 Figure 안에서 서브플롯(subplot)(아래 그림에서 Axes에 해당)의 위치를 조정할 때 사용함. left, bottom, right, top으로 이미지 위치를 조정함. 또한 hspace와 wspace를 사용해 서브플롯 간의 간격(너비와 높이의 비율)을 조정 가능.



5.3.2 미세 조정 기법

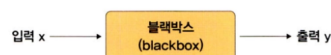


- 미세 조정(fine-tuning) 기법은 특성 추출 기법에서 더 나아가 사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련시키는 방식.
특성 추출은 목표 특성을 잘 추출했다는 전제 하에 좋은 성능을 낼 수 있음. 특성 추출이 잘못되었다면 (e.g. ImageNet 데이터셋의 이미지 특징과 전자상거래 물품의 이미지 특징이 다를 경우) fine-tuning 기법으로 새로운 (전자상거래) 이미지 데이터를 사용해 네트워크의 가중치를 업데이트해 특성을 다시 추출할 수 있음.
즉, pre-trained 모델을 목적에 맞게 재학습시키거나 학습된 가중치 일부를 재학습시키는 것.
- Fine-tuning 기법은 사전 훈련된 네트워크를 미세 조정하여 분석하려는 데이터셋에 잘 맞도록 모델의 파라미터를 조정하는 기법. Fine-tuning 과정에서 많은 연산량이 필요하기 때문에 CPU보다는 GPU 사용 권장
- Fine-tuning 기법 전략 (훈련시키려는 데이터셋 크기와 사전 훈련된 모델에 따라서)

	데이터셋이 큼	데이터셋이 작음
사전 훈련된 모델과 유사성이 큼	합성곱층의 뒷부분(완전연결층과 가까운 부분)과 데이터 분류기를 학습시킨다. (데이터셋이 유사하기 때문)	데이터 분류기만 학습시킨다. 데이터가 적기 때문에 많은 계층에 fine-tuning 적용하면 과적합 발생 가능.
사전 훈련된 모델과 유사성이 작음	모델 전체를 재학습시킨다.	합성곱층의 일부분과 데이터 분류기를 학습시킨다. (합성곱층 중 어디까지 새로 학습시켜야 할지 적당히 설정해야 함)

- 미세 조정은 파라미터 업데이트 과정에서 파라미터에 큰 변화를 주게 되면 과적합 문제가 발생할 수 있기 때문에, 정교하고 미세한 파라미터 업데이트가 필요함.

5.4 설명 가능한 CNN



설명 가능한 CNN(explainable CNN)은 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술.
(by. CNN 처리과정 시각화)

5.4.1 특성 맵 시각화

특성맵(feature map) 혹은 활성화 맵은 입력 이미지 또는 다른 특성 맵처럼 필터를 입력에 적용한 결과. 따라서 특정 입력 이미지에 대한 feature map을 시각화한다는 의미는 feature map에서 입력 특성을 감지하는 방법을 이해할 수 있도록 돕는것.

특성 맵 시각화 예제 코드

▼ 필요한 라이브러리 PIL(Python Image Library) 설치

이미지 분석 및 처리를 쉽게 할 수 있도록 도와주는 PIL 라이브러리 설치.

PIL은 다양한 이미지 파일 형식을 지원하며, 강력한 이미지 처리와 그래픽 기능을 제공하는 이미지 프로세싱 라이브러리.

```
pip install pillow
```

```
# 필요한 라이브러리 호출
import matplotlib.pyplot as plt
from PIL import Image
import cv2
import torch
import torch.nn.functional as F
import torch.nn as nn
from torchvision.transforms import ToTensor
import torchvision
import torchvision.transforms as transforms
import torchvision.models as models

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
```

설명 가능한 모델을 위해, 13개의 합성곱층과 두 개의 완전연결층으로 구성된 네트워크 생성. (합성곱층과 완전연결층은 ReLU 사용)

```
# 설명 가능한 네트워크 생성
class XAI(torch.nn.Module):
    def __init__(self, num_classes=2):
        super(XAI, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True), # inplace=True는 기존의 데이터를 연산의 결과값으로 대체하
            는 것을 의미
            nn.Dropout(0.3),
            nn.Conv2d(64, 64, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Conv2d(128, 128, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=2, stride=2),

            nn.Conv2d(128, 256, kernel_size=3, padding=1, bias=False),
```

```

        nn.BatchNorm2d(256),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(256),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Conv2d(256, 256, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(256),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2),

        nn.Conv2d(256, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2),

        nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Conv2d(512, 512, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(512),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(kernel_size=2, stride=2),
    )
    self.classifier = nn.Sequential(
        nn.Linear(512, 512, bias=False),
        nn.Dropout(0.5),
        nn.BatchNorm1d(512),
        nn.ReLU(inplace=True),
        nn.Dropout(0.5),
        nn.Linear(512, num_classes)
    )

    def forward(self, x) :
        x = self.features(x)
        x = x.view(-1, 512)
        x = self.classifier(x)
        return F.log_softmax(x) # 설명

```

- `log_softmax()` 는 신경망 말단의 결과값들을 확률 개념으로 해석하기 위해 softmax 함수의 결과에 log 값을 취한 연산. softmax가 아닌 log softmax를 사용하는 이유는, softmax는 기울기 소멸 문제에 취약하기 때문임.

다음은 softmax와 log softmax에 대한 수식이다.

- $softmax(x)_i = \frac{e^{x_i}}{\sum_{j=1} e^{x_j}}$
- $logsoftmax = \log(\frac{e^{x_i}}{\sum_{j=1} e^{x_j}}) = x_i - \log(\sum_{j=1} e^{x_j})$

```
# 모델 객체화
model = XAI() # model이라는 이름의 객체 생성
model.to(device) # model을 device에 할당
model.eval() # 테스트 데이터에 대한 모델 평가 용도로 사용
```

▼ 코드 실행 결과

```
XAI(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), bias=F
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, t
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.3, inplace=False)
    (4): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), paddi
    (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, t
    (6): ReLU(inplace=True)
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padd
    (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
    (10): ReLU(inplace=True)
    (11): Dropout(p=0.4, inplace=False)
    (12): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), pa
    (13): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
    (14): ReLU(inplace=True)
    (15): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation
    (16): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), pa
    (17): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
    (18): ReLU(inplace=True)
    (19): Dropout(p=0.4, inplace=False)
    (20): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), pa
    (21): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
    (22): ReLU(inplace=True)
    ...
    (3): ReLU(inplace=True)
    (4): Dropout(p=0.5, inplace=False)
    (5): Linear(in_features=512, out_features=2, bias=True)
  )
)
```

특성 맵은 합성곱층을 입력 이미지와 필터를 연산하여 얻은 결과 → 합성곱층에서 입력과 출력을 알 수 있다면, 특성 맵에 대한 값들을 확인할 수 있음.

```
# 특성 맵을 확인하기 위한 클래스 정의
class LayerActivations:
    features = []
    def __init__(self, model, layer_num):
        self.hook = model[layer_num].register_forward_hook(self.hook_fn) # 설명

    def hook_fn(self, module, input, output):
        self.features = output.detach().cpu().numpy()

    def remove(self): # hook 삭제
        self.hook.remove()
```

- `self.hook`

파이토치는 매 계층마다 print문을 사용하지 않더라도 hook 기능을 사용해 각 계층의 활성화 함수 및 기울기 값 (즉, 중간 결과 값) 을 확인할 수 있음. 따라서 `register_forward_hook` 의 목적은 순전파 중 각 네트워크 모듈의 입력 및 출력을 가져오는 것.

▼ e.g.

아래의 코드에 대한 출력 결과

- 코드

```
import torch
x = torch.Tensor([0,1,2,3]).requires_grad_()
y = torch.Tensor([4,5,6,7]).requires_grad_()
w = torch.Tensor([1,2,3,4]).requires_grad_()
z = x + y;
o = w.matmul(z)
o.backward()
print(x.grad, y.grad, z.grad, w.grad, o.grad)
```

- 실행 결과

```
tensor([2., 3., 4., 5.]) tensor([2., 3., 4., 5.]) None tensor([ 4.,  6.,  8., 10.]) None
```

코드에서 o와 z는 특정한 값으로 정의되지 않은 중간 변수(계산 결과에 따라 값이 달라질 수 있는 변수). 파이토치는 이러한 변수에 대해서는 기울기 값을 저장하지 않음. 하지만 이와 같은 중간 변수에 대해 `r.register_hook(hook_fn)` 을 사용하면 기울기 값을 알 수 있음.

```
# 이미지 호출
img = cv2.imread("/Users/eunseoshin/DL-pytorch/chap05/data/cat.jpg")
plt.imshow(img)
img = cv2.resize(img, (100, 100), interpolation=cv2.INTER_LINEAR) # 설명(1)
img = ToTensor()(img).unsqueeze(0).to(device) # 설명(2)
print(img.shape)
```

▼ 코드 실행 결과

```
torch.Size([1, 3, 100, 100])
```



- 설명 (1)

`cv2.resize` 는 이미지 크기를 변경할 때 사용하며, 파라미터는 다음과 같다.

```
cv2.resize(img, (100, 100), interpolation=cv2.INTER_LINEAR)
```

- 첫 번째 파라미터 : 변경할 이미지 파일
- 두 번째 파라미터 : 변경될 이미지 크기를 (너비, 높이)로 지정
- `interpolation` : 보간법

이미지 크기를 변경할 경우, 변형된 이미지의 픽셀을 추정해서 값을 할당해야 함. 이미지 비율을 변경하면 존재하지 않는 영역에 새로운 픽셀 값을 매핑하거나 존재하는 픽셀들을 압축해 새로운 값을 할당해야 함.

보간법이란? 위와 같은 상황을 피하고자 이미지에 존재하는 픽셀 데이터 (x_i, y_i) 들에 대해 근사 함수 $f(x, y)$ 를 적용해 새로운 픽셀 값을 구하는 것.

즉, 추정해야 하는 픽셀을 보간법을 이용해 값을 할당함.

- 설명 (2)

`ToTensor()(img).unsqueeze(0)` 에서 사용된 `unsqueeze()` 는 1차원 데이터를 생성하는 함수. 즉, 이미지 데이터를 텐서로 변환하고, 그 변환된 데이터를 1차원으로 변경하겠다는 의미.

- ▼ e.g. unsqueeze 사용 예시

- 코드

```
import torch

x1 = torch.rand(3, 10, 64)

x2 = x1.unsqueeze(dim=0) ----- [3, 10, 64] → [1, 3, 10, 64]
print(x2.shape)
print('-----')
x3 = x1.unsqueeze(dim=1) ----- [3, 10, 64] → [3, 1, 10, 64]
print(x3.shape)
```

- 실행 결과

```
torch.Size([1, 3, 10, 64])
-----
torch.Size([3, 1, 10, 64])
```

앞 코드(네트워크 생성 셀)의 출력 결과인 (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), bias=False)에 대한 특성 맵 확인해보기

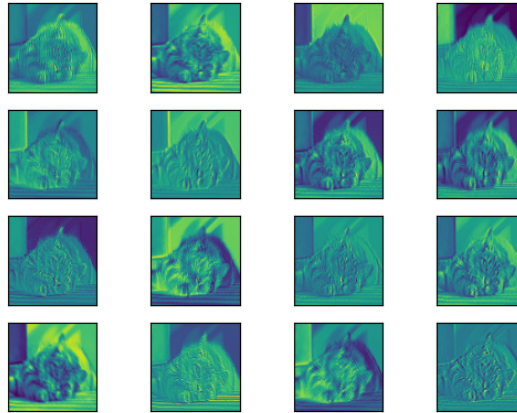
```
# (0): Conv2d feature map 확인
result = LayerActivations(model.features, 0) # 0번째 Conv2d feature map 확인

model(img)
activations = result.features
```

```
# feature map 확인
fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize=(12,8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)

for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
```

- ▼ 코드 실행 결과



첫 번째 Conv2d 계층에서 feature map에 대한 출력 결과.

입력층에 가까운 계층으로 입력 이미지의 형태가 많이 유지되고 있음.

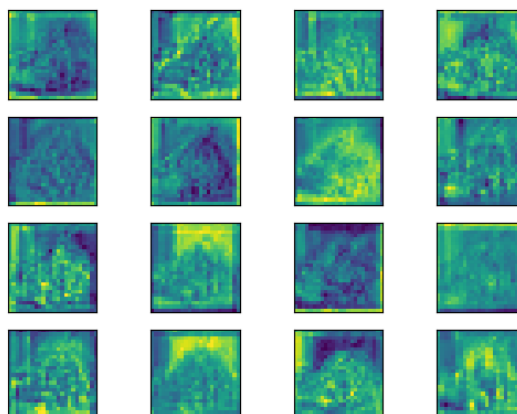
```
# 20번째 계층에 대한 feature map
result = LayerActivations(model.features, 20) # 20번째 Conv2d feature map 확인

model(img)
activations = result.features

# feature map 확인
fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize=(12,8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)

for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
```

▼ 코드 실행 결과



20번째 feature map의 결과 기존 고양이 이미지의 형태를 찾아보기 어려움.

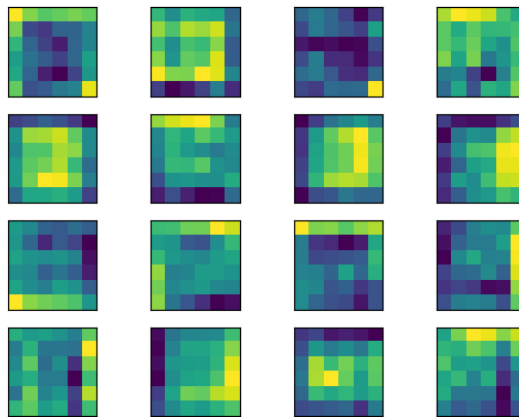
```
# 40번째 계층에 대한 feature map
result = LayerActivations(model.features, 40) # 40번째 Conv2d feature map 확인

model(img)
activations = result.features

# feature map 확인
fig, axes = plt.subplots(4, 4)
fig = plt.figure(figsize=(12,8))
fig.subplots_adjust(left=0, right=1, bottom=0, top=1, hspace=0.05, wspace=0.05)

for row in range(4):
    for column in range(4):
        axis = axes[row][column]
        axis.get_xaxis().set_ticks([])
        axis.get_yaxis().set_ticks([])
        axis.imshow(activations[0][row*10+column])
plt.show()
```

▼ 코드 실행 결과



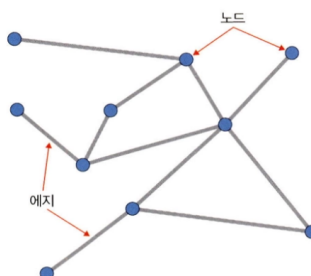
원래 입력 이미지에 대한 형태는 전혀 찾아볼 수 없음. 즉, 출력층에 가까울수록 원래 형태는 찾아볼 수 없고, 이미지 특징들만 전달되는 것을 확인 가능.

5.5 그래프 합성곱 네트워크

그래프 합성곱 네트워크(graph convolutional network)는 그래프 데이터를 위한 신경망.

5.5.1 그래프란

- 그래프 : 방향성이 있거나(directed) 없는(undirected) edge로 연결된 노드(nodes=vertices)의 집합



- 여기에서, 노드와 에지는 일반적으로 풀고자 하는 문제에 대한 전문가 지식이나 직관으로 구성됨.
- 그래프의 구성 요소
 - 노드(node, vertex) : 위 그림에서 파란색 원
 - 에지(edge) : 두 노드를 연결한 선
- 즉, 노드는 원소들, 에지는 결합 방법(single, double, triple, aromatic 등)을 의미

5.5.2 그래프 신경망

- 그래프 신경망(Graph Neural Network, GNN) : 그래프 구조에서 사용하는 신경망
- 그래프 데이터에 대한 표현 (두 단계)



1단계. 인접 행렬(adjacency matrix)

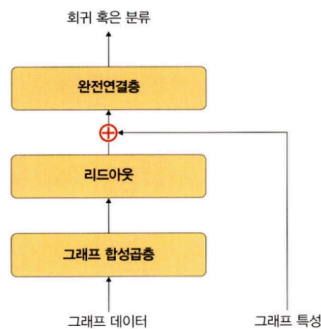
- 위 그림의 왼쪽과 같은 네트워크가 있을 때 노드 n 개를 $n \times n$ 행렬(matrix)로 표현함.
- 이렇게 생성된 인접 행렬 내의 값은 ' A_{ij} 는 i 와 j 의 관련성 여부'를 만족하는 값으로 채워짐.

2단계. 특성 행렬(feature matrix)

- 인접 행렬만으로는 특성을 파악하기 어렵기 때문에 단위 행렬을 적용함.
- 각 입력 데이터에서 이용할 특성을 선택.
- 특성 행렬에서 각 행은 선택된 특성에 대해 각 노드가 갖는 값을 의미 (e.g. 첫 번째 행은 첫 번째 노드의 특성 값)
- 이제 노드 1 · 2 · 3 · 4 에 대한 특성을 한눈에 파악하기 쉽게 표현됨.

5.5.3 그래프 합성곱 네트워크

- 그래프 합성곱 네트워크(Graph Convolutional Network, GCN) : 이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘
- 그래프 합성곱 네트워크의 구조



- 리드아웃(readout) : 특성 행렬을 하나의 벡터로 변환하는 함수. 즉, 전체 노드의 특성 벡터에 대해 평균을 구하고 그래프 전체를 표현하는 하나의 벡터 생성.
- 그래프 합성곱층(graph convolutional layer)

GCN에서 가장 중요한 부분. 그래프 합성곱층을 이용한 그래프 형태의 데이터는 행렬 형태의 데이터로 변환되어 딥러닝 알고리즘을 적용할 수 있기 때문. (기존 그래프 형태의 데이터로는 딥러닝 알고리즘을 적용할 수 없음)
- GCN의 활용 : SNS에서 관계 네트워크, 학술 연구에서 인용 네트워크, 3D Mesh